

```
"""Datenmodelle für das Dreigliederungs-
Simulationsspiel (Arbeitstitel: Korn-
Spiel).
```

```
Grundsätze:
```

```
- Pydantic v2 an den Rändern: Missionen
(YAML), Absichten (Netzwerk), Spielersicht.
- Schlanke dataclass für die Korn-Tranche
im Kern, weil der Ledger sehr viele davon
bewegt.
- Unabhängig von Mesa: Die Mesa-Agenten
tragen diese Datensätze, nicht umgekehrt.
  So bleibt das Modell stabil, auch wenn
sich die Mesa-API (Mesa 4) ändert.
"""
```

```
from __future__ import annotations
```

```
import math
from dataclasses import dataclass
from enum import Enum
from typing import Annotated, ClassVar,
Literal, Union
```

```
from pydantic import BaseModel, ConfigDict,
Field, model_validator
```

```
# === ABSCHNITT grundbegriffe ===
```

```
HALME_JE_KORN = 100 # 1 Korn = 100 Halme;
intern wird nur ganzzahlig gerechnet
```

```
TAGE_JE_RUNDE = 91    # eine Runde = eine
Jahreszeit
```

```
Halme = Annotated[int, Field(ge=0)]
Id = Annotated[str, Field(min_length=1,
pattern=r"^[a-z0-9][a-z0-9_\-]*$")]
```

```
class Sphaere(str, Enum):
    """Die drei Glieder des sozialen
    Ökosystems (Gewaltenteilung 2.0)."""

    KULTUR = "kultur"          #
    Kultursphäre
    RECHT = "recht"           # Politik
    und Recht
    WIRTSCHAFT = "wirtschaft" #
    Wirtschaftsleben
```

```
class Modell(BaseModel):
    """Basis: unbekannte Felder sind ein
    Fehler, keine stille Überraschung."""

    model_config =
    ConfigDict(extra="forbid")
    # === ENDE ===
```

```
# === ABSCHNITT korn ===
class Verwendung(str, Enum):
    """Kein Sortenmerkmal des Geldes,
    sondern der Zustand, in dem es gebraucht
    wird."""
```

```

    KAUF = "kauf"
    LEIH = "leih"
    SCHENK = "schenk"

@dataclass(slots=True)
class Tranche:
    """Eine Korn-Tranche. Korn ist nicht
    fungibel: Betrag UND Ablaufdatum zählen."""

    id: int
    halme: int
    ablauf_tag: int
    halter: str
    region: str
    verwendung: Verwendung =
Verwendung.KAUF
    geboren_tag: int = 0

    def restlaufzeit(self, tag: int) ->
int:
        return self.ablauf_tag - tag

class KornRegeln(Modell):
    """Alle Stellschrauben des Protokolls
    an einem Ort."""

    frist_jahre: float = Field(20, gt=0)
    auffrischung_ab_jahren: float =
Field(1, gt=0)
    schenkgrenze_tage: int = Field(90,
ge=0)

```

```

        emission: Literal["nur_auffrischung",
"ertragsgedeckt"] = "nur_auffrischung"
        emission_je_ertrag: float = Field(0.0,
ge=0)

        @model_validator(mode="after")
        def _plausibel(self) -> "KornRegeln":
            if self.auffrischung_ab_jahren >=
self.frist_jahre:
                raise ValueError("Auffrischung
muss vor Ablauf der Frist beginnen")
                if self.schenkgrenze_tage >=
self.frist_jahre * 365:
                    raise ValueError("Schenkgrenze
länger als die ganze Frist")
                    if self.emission ==
"ertragsgedeckt" and
self.emission_je_ertrag == 0:
                        raise
ValueError("ertragsgedeckte Emission
braucht emission_je_ertrag > 0")
                    return self
            # === ENDE ===

# === ABSCHNITT region ===
class Naturgut(Modell):
    """Ein Posten der Naturgrundlage: der
nachwachsende Teil, durch Arbeit
gewonnen."""

    name: Id
    normalertrag: float = Field(gt=0)
    gewicht: float = Field(1.0, gt=0)

```

```

    einheit: str = "t"

class Naturgrundlage(Modell):
    """Das regionale Portfolio. Sein Index
    steuert die Kaufkraft, nicht die
    Geldmenge."""

    gueter: list[Naturgut] =
    Field(min_length=1)

    def index(self, ertraege: dict[str,
    float]) -> float:
        summe = sum(g.gewicht for g in
    self.gueter)
        return sum(
            g.gewicht *
    ertraege.get(g.name, g.normalertrag) /
    g.normalertrag
            for g in self.gueter
        ) / summe

class EuroStelle(Modell):
    """Die Membran zur Außenwelt. Euro nie
    nach innen, Korn nie nach außen.

    Euro kommen nur als eingebrachte
    Produktionsmittel herein. Mitglieder zahlen
    hier Korn, die Stelle bestellt draußen
    in Euro - und umgekehrt.
    """

    id: Id

```

```

        euro_vorrat: float = Field(0.0, ge=0)

class Region(Modell):
    id: Id
    name: str
    naturgrundlage: Naturgrundlage
    euro_stelle: EuroStelle | None = None
    rechenstelle: Id = "welt" # regionale
    Portfolios fließen in die Weltrechnung
    # === ENDE ===

# === ABSCHNITT menschen ===
class Steuerung(str, Enum):
    MENSCH = "mensch"
    BOT = "bot"

class Rolle(str, Enum):
    """Aus welcher Kraft heraus eine Person
    gerade handelt."""

    BUERGER = "buerger" # jede Person,
    immer
    EVG = "evg" # als Organ des
    Wirtschaftslebens
    RECHT = "recht" # als Organ von
    Politik und Recht
    KULTUR = "kultur" # als Organ der
    Kultursphäre

ROLLEN_SPHAERE: dict[Rolle, Sphaere | None]

```

```

= {
    Rolle.BUERGER: None,
    Rolle.EVG: Sphaere.WIRTSCHAFT,
    Rolle.RECHT: Sphaere.RECHT,
    Rolle.KULTUR: Sphaere.KULTUR,
}

class Mitgliedschaft(Modell):
    """Eine Beziehung zu einem Organ.
    Mitgliedschaften dürfen sich überlappen."""

    organ: Id
    sphaere: Sphaere
    funktion: str = "mitglied" # z. B.
    erzeuger, verwender, lehrerin,
    antragstellerin

class Beduerfnisse(Modell):
    """Aufbauphase: erst der
    Lebensunterhalt, dann die Arbeitsmittel in
    Korn."""

    lebensunterhalt_je_runde: Halme
    arbeitsmittel_je_runde: Halme = 0

    @property
    def je_tag(self) -> float:
        return
        (self.lebensunterhalt_je_runde +
        self.arbeitsmittel_je_runde) /
        TAGE_JE_RUNDE

```

```

class Person(Modell):
    """Ein Mensch - gespielt oder
    simuliert.

    Die drei Sphären sind keine drei
    Gruppen von Menschen, sondern drei
    Beziehungen jedes Menschen. Deshalb hat
    eine Person Mitgliedschaften
    in allen Sphären zugleich, aber genau
    eine Rolle, die sie gerade spielt.
    """

    id: Id
    name: str
    region: Id
    steuerung: Steuerung = Steuerung.BOT
    spielt_rolle: Rolle = Rolle.BUERGER
    mitgliedschaften: list[Mitgliedschaft]
    = []
    beduerfnisse: Beduerfnisse
    faehigkeiten: list[str] = []

    @model_validator(mode="after")
    def _rolle_braucht_organ(self) ->
    "Person":
        sphaere =
        ROLLEN_SPHAERE[self.spielt_rolle]
        if sphaere and not any(m.sphaere is
        sphaere for m in self.mitgliedschaften):
            raise ValueError(
                f"{self.id}: Rolle
                {self.spielt_rolle.value} ohne
                Mitgliedschaft "

```



```

        f"in einem Organ der Sphäre
{sphaere.value}"
    )
    return self
# === ENDE ===

# === ABSCHNITT organe ===
class Organ(Modell):
    id: Id
    name: str
    region: Id

class EVG(Organ):
    """Erzeuger-Verwender-Gemeinschaft:
    findet Preise, setzt sie nicht per
    Macht."""

    sphaere: Literal[Sphaere.WIRTSCHAFT] =
Sphaere.WIRTSCHAFT
    produkte: list[Id] =
Field(min_length=1)
    erzeuger: list[Id] =
Field(min_length=1)
    haendler: list[Id] = []
    verwender: list[Id] =
Field(min_length=1)
    max_mitglieder: int = Field(20, gt=0)
# Zwanziger-Regel als Schalter

@property
def mitglieder(self) -> set[str]:
    return set(self.erzeuger) |

```

```

set(self.haendler) | set(self.verwender)

    @model_validator(mode="after")
    def _groesse(self) -> "EVG":
        if len(self.mitglieder) >
self.max_mitglieder:
            raise ValueError(f"{self.id}:
mehr als {self.max_mitglieder} Mitglieder")
        return self

class Naturbetrieb(Organ):
    """Der einzige Ort, an dem Korn
entsteht oder aufgefrischt wird."""

    sphaere: Literal[Sphaere.WIRTSCHAFT] =
Sphaere.WIRTSCHAFT
    naturgut: Id
    betreiber: list[Id] =
Field(min_length=1)
    anerkannt: bool = False

class Rahmenregel(Modell):
    """Was Politik und Recht beschließen
darf: Rahmen, keine Preise, keine
Inhalte."""

    id: Id
    titel: str
    gegenstand: Literal["arbeitszeit",
"bodenrecht", "schutz", "verfahren",
"sonstiges"]
    werte: dict[str, float | str] = {}

```

```

        beschlossen_runde: int | None = None

class Rechtsgemeinschaft(Organ):
    """Alle mündigen Menschen der Region,
    gleichberechtigt. Das Organ bereitet
    vor."""

    sphaere: Literal[Sphaere.RECHT] =
Sphaere.RECHT
    regeln: list[Rahmenregel] = []
    mehrheit: float = Field(0.5, gt=0,
le=1)

class Kulturstaeette(Organ):
    """Schule, Forschung, Kunst, Heilkunst
    - lebt von Schenkgeld."""

    sphaere: Literal[Sphaere.KULTUR] =
Sphaere.KULTUR
    art: Literal["schule", "forschung",
"kunst", "heilkunst", "sonstiges"]
    mitwirkende: list[Id] =
Field(min_length=1)
    bedarf_je_runde: Halme
# === ENDE ===

# === ABSCHNITT preis ===
class Produkt(Modell):
    id: Id
    name: str
    herstellungsdauer_tage: int =

```

```

Field(gt=0)
    menge_je_herstellung: float =
Field(gt=0)
    einheit: str = "stück"

def gerechter_preis(erzeuger: Person,
produkt: Produkt) -> int:
    """Urzelle des Wirtschaftslebens (GA
340), als Rechenregel.

    Der Erzeuger soll für sein Erzeugnis so
viel erhalten, dass er seine
    Bedürfnisse decken kann, bis er ein
gleiches Erzeugnis wieder hergestellt hat.
    Ergebnis in Halmen je Einheit,
aufgerundet.
    """
    bedarf = erzeuger.beduerfnisse.je_tag *
produkt.herstellungsdauer_tage
    return math.ceil(bedarf /
produkt.menge_je_herstellung)

class Preisvereinbarung(Modell):
    """Ergebnis einer EVG-Verhandlung. Der
gerechte Preis steht als Maßstab
daneben."""

    evg: Id
    produkt: Id
    preis_je_einheit: Halme
    gerechter_preis: Halme
    gueltig_ab_runde: int = Field(ge=0)

```

```

    @property
    def abweichung(self) -> float:
        """Relative Abweichung vom
gerechten Preis: 0 = gerecht, <0 = Erzeuger
verarmt."""
        if self.gerechter_preis == 0:
            return 0.0
        return self.preis_je_einheit /
self.gerechter_preis - 1.0
# === ENDE ===

# === ABSCHNITT absichten ===
class Absicht(Modell):
    """Was jemand will. Ob es gilt,
entscheidet allein die Welt (der Server).

    `ziel_sphaere` ist gesetzt, wenn die
Handlung eine Sphäre gestaltet.
    Bürgerhandlungen (zahlen, schenken,
abstimmen) gestalten keine - der
    Mensch nimmt an allen drei Sphären
teil, ohne überzugreifen.
    """

    ziel_sphaere: ClassVar[Sphaere | None]
= None
    von: Id
    runde: int = Field(ge=0)
    als_rolle: Rolle = Rolle.BUERGER

class Zahlen(Absicht):

```

```

    art: Literal["zahlen"] = "zahlen"
    an: Id
    halme: Halme
    zweck: str = ""

class Schenken(Absicht):
    art: Literal["schenken"] = "schenken"
    an: Id
    halme: Halme

class Abstimmen(Absicht):
    art: Literal["abstimmen"] = "abstimmen"
    antrag: Id
    stimme: Literal["ja", "nein",
"enthaltung"]

class BedarfMelden(Absicht):
    art: Literal["bedarf_melden"] =
"bedarf_melden"
    produkt: Id
    menge: float = Field(gt=0)

class VorhabenEinbringen(Absicht):
    """Leihgeld suchen. Knapp ist nicht der
Betrag, sondern die lange Frist."""

    ziel_sphaere: ClassVar[Sphaere | None]
= Sphaere.WIRTSCHAFT
    art: Literal["vorhaben"] = "vorhaben"
    halme: Halme

```

```

    mindestfrist_jahre: float = Field(gt=0)
    titel: str = ""

class Importieren(Absicht):
    ziel_sphaere: ClassVar[Sphaere | None]
= Sphaere.WIRTSCHAFT
    art: Literal["importieren"] =
"importieren"
    euro_stelle: Id
    produkt: Id
    menge: float = Field(gt=0)

# --- die drei gestaltenden Handlungen: je
eine pro Sphäre ---

class PreisVorschlagen(Absicht):
    ziel_sphaere: ClassVar[Sphaere | None]
= Sphaere.WIRTSCHAFT
    art: Literal["preis_vorschlagen"] =
"preis_vorschlagen"
    evg: Id
    produkt: Id
    preis_je_einheit: Halme

class RegelBeantragen(Absicht):
    ziel_sphaere: ClassVar[Sphaere | None]
= Sphaere.RECHT
    art: Literal["regel_beantragen"] =
"regel_beantragen"
    rechtsgemeinschaft: Id
    regel: Rahmenregel

```

```

class InhaltBestimmen(Absicht):
    """Was gelehrt, erforscht, gestaltet
    wird."""

    ziel_sphaere: ClassVar[Sphaere | None]
    = Sphaere.KULTUR
    art: Literal["inhalt_bestimmen"] =
    "inhalt_bestimmen"
    kulturstaette: Id
    inhalt: str

class UmSchenkungWerben(Absicht):
    ziel_sphaere: ClassVar[Sphaere | None]
    = Sphaere.KULTUR
    art: Literal["werben"] = "werben"
    kulturstaette: Id
    botschaft: str

AbsichtBeliebig = Annotated[
    Union[
        Zahlen, Schenken, Abstimmen,
        BedarfMelden, VorhabenEinbringen,
        Importieren,
        PreisVorschlagen, RegelBeantragen,
        InhaltBestimmen, UmSchenkungWerben,
    ],
    Field(discriminator="art"),
]

```



```

class AbsichtsPaket(Modell):
    """Was ein Client pro Runde an den
    Server schickt."""

    person: Id
    runde: int = Field(ge=0)
    absichten: list[AbsichtBeliebig] = []
# === ENDE ===

# === ABSCHNITT uebergreif ===
class Uebergreif(Modell):
    """Ein Organ handelt mit der Kraft
    seiner Sphäre in einer fremden."""

    runde: int
    person: Id
    quelle: Sphaere
    ziel: Sphaere
    absicht: str

    @property
    def zelle(self) -> str:
        return f"{self.quelle.value}->
{self.ziel.value}"

def klassifiziere(a: Absicht) -> Uebergreif
| None:
    """Rolle × gestaltende Handlung ergibt
    genau die 3×3-Übergriffs-Matrix.

    Diagonale = gesund. Die sechs Zellen
    daneben = Übergriffe.

```

Erlaubt sind sie trotzdem: Die Folgen
zeigt die Simulation.

```
"""
    quelle = ROLLEN_SPHAERE[a.als_rolle]
    ziel = type(a).ziel_sphaere
    if quelle is None or ziel is None or
    quelle is ziel:
        return None
    return Uebergriff(runde=a.runde,
    person=a.von, quelle=quelle, ziel=ziel,

absicht=type(a).__name__)
# === ENDE ===

# === ABSCHNITT runde ===
class Jahreszeit(str, Enum):
    FRUEHLING = "frühling"
    SOMMER = "sommer"
    HERBST = "herbst"
    WINTER = "winter"

class Phase(str, Enum):
    NATUR = "natur" # Welt:
    Ernte, Alterung, Auffrischung
    ABSICHTEN = "absichten" #
    Menschen und Bots geben Absichten ab
    VERHANDLUNG = "verhandlung" # EVG
    einigen sich auf Preise
    ABSTIMMUNG = "abstimmung" #
    Rechtsgemeinschaft entscheidet
    AUFLOESUNG = "aufloesung" # Welt
    prüft, bucht, klassifiziert Übergriffe
```

```

RUECKSCHAU = "rueckschau"      #
Kennzahlen und Folgen werden sichtbar

class Runde(Model):
    nr: int = Field(ge=0)
    phase: Phase = Phase.NATUR

    @property
    def jahr(self) -> int:
        return self.nr // 4 + 1

    @property
    def jahreszeit(self) -> Jahreszeit:
        return list(Jahreszeit)[self.nr %
4]

    @property
    def erster_tag(self) -> int:
        return self.nr * TAGE_JE_RUNDE

class Kennzahlen(Model):
    """Die Wahrnehmungsorgane des Spiels -
für alle sichtbar."""

    runde: int
    deckung_lebensunterhalt: float =
Field(ge=0, le=1)
    preisabweichung_median: float
    geldmenge: Halme
    schenkgeld_zur_kultur: Halme
    fristen_mismatch_tage: int =
Field(ge=0)

```

```

        uebergriffe: dict[str, int] = {} #
Matrixzelle -> Anzahl in dieser Runde
        beteiligung_abstimmung: float =
Field(ge=0, le=1)
# === ENDE ===

# === ABSCHNITT sicht ===
class KontoSicht(Modell):
    """Nur das eigene Konto. Fremde
Buchungen bleiben verborgen, wie im Korn-
Protokoll."""

    gesamt: Halme
    nach_verwendung: dict[Verwendung,
Halme]
    davon_sterbend: Halme
    naechster_ablauf_tag: int | None

class Antrag(Modell):
    id: Id
    regel: Rahmenregel
    antragsteller: Id
    als_rolle: Rolle
    offen_bis_runde: int

class Sicht(Modell):
    """Was der Server einem Client pro
Phase schickt."""

    runde: Runde
    person: Id

```

```

    rolle: Rolle
    konto: KontoSicht
    offene_antraege: list[Antrag] = []
    preise: list[Preisvereinbarung] = []
    kennzahlen: Kennzahlen | None = None
    meldungen: list[str] = []
# === ENDE ===

# === ABSCHNITT mission ===
class Schock(Modell):
    naturgut: Id | Literal["*"]
    ab_runde: int = Field(ge=0)
    dauer_runden: int = Field(4, gt=0)
    faktor: float = Field(ge=0)

class Mission(Modell):
    """Ein Szenario als Spielmission: Welt,
    Menschen, Organe, Schocks - und eine
    Frage."""

    id: Id
    titel: str
    frage: str
    runden: int = Field(gt=0)
    seed: int = 42
    korn: KornRegeln = KornRegeln()
    regionen: list[Region] =
Field(min_length=1)
    produkte: list[Produkt] = []
    personen: list[Person] =
Field(min_length=1)
    evgs: list[EVG] = []

```

```

        naturbetriebe: list[Naturbetrieb] =
Field(min_length=1)
        rechtsgemeinschaften:
list[Rechtsgemeinschaft] =
Field(min_length=1)
        kulturstaetten: list[Kulturstaette] =
[]
        schocks: list[Schock] = []
        startkorn: dict[Id, Halme] = {} #
Halter -> Halme

@model_validator(mode="after")
def _zusammenhang(self) -> "Mission":
    fehler: list[str] = []
    organe = [*self.evgs,
*self.naturbetriebe,

*self.rechtsgemeinschaften,
*self.kulturstaetten]
        alle_ids = [p.id for p in
self.personen] + [o.id for o in organe]
        doppelt = {i for i in alle_ids if
alle_ids.count(i) > 1}
        if doppelt:
            fehler.append(f"doppelte Ids:
{sorted(doppelt)}")

        regionen = {r.id for r in
self.regionen}
        personen = {p.id for p in
self.personen}
        organ_sphaere = {o.id: o.sphaere
for o in organe}
        produkte = {p.id for p in

```

```

self.produkte}
        gueter = {g.name for r in
self.regionen for g in
r.naturgrundlage.gueter}

        for x in [*self.personen, *organe]:
            if x.region not in regionen:
                fehler.append(f"{x.id}:
unbekannte Region {x.region}")
            for p in self.personen:
                for m in p.mitgliedschaften:
                    if m.organ not in
organ_sphaere:
                        fehler.append(f"{p.id}:
unbekanntes Organ {m.organ}")
                        elif organ_sphaere[m.organ]
is not m.sphaere:
                            fehler.append(f"{p.id}:
{m.organ} gehört nicht zur Sphäre
{m.sphaere.value}")
                    for e in self.evgs:
                        for pid in e.mitglieder -
personen:
                            fehler.append(f"{e.id}:
unbekannte Person {pid}")
                        for prod in set(e.produkte) -
produkte:
                            fehler.append(f"{e.id}:
unbekanntes Produkt {prod}")
                        for n in self.naturbetriebe:
                            if n.naturgut not in gueter:
                                fehler.append(f"{n.id}:
Naturgut {n.naturgut} fehlt in der
Naturgrundlage")

```

```

        if not any(n.anerkannt for n in
self.naturbetriebe):
            fehler.append("kein anerkannter
Naturbetrieb - Korn könnte nie aufgefrischt
werden")
            for r in self.regionen:
                if not any(rg.region == r.id
for rg in self.rechtsgemeinschaften):
                    fehler.append(f"Region
{r.id} ohne Rechtsgemeinschaft")
                    for halter in self.startkorn:
                        if halter not in alle_ids:
                            fehler.append(f"Startkorn
für unbekannten Halter {halter}")
                    for s in self.schocks:
                        if s.naturgut != "*" and
s.naturgut not in gueter:
                            fehler.append(f"Schock auf
unbekanntes Naturgut {s.naturgut}")
                    if fehler:
                        raise ValueError(";
".join(fehler))
            return self
# === ENDE ===

```